

# Projet 3DOKR

Dans ce document vous retrouverez, toutes les informations à avoir concernant le projet Docker 3DOKR.

Avant de commencer, nous avons eu recours à quelque modification dans les fichiers de configuration de l'application donnée au préalable. Nous avons du modifier les chemins permettant de relier les composants entre eux, afin que la base de donnée puisse être utilisable.

## 1. Documentation Dockerfiles

Pour chaque service de notre application, nous avons créé un Dockerfile permettant de construire une image personnalisée. Voici les commandes à faire pour utiliser ses fichiers :

```
docker build -t python-app:1.0 ./python-app
docker build -t node:1.0 ./node
docker build -t worker:1.0 ./dotnet
docker build -t redis:1.0 ./redis
docker build -t postgresql:1.0 ./postgresql
```

Afin que tout fonctionne correctement, vous devrez créer un network afin que les conteneurs communiquent entre eux :

```
docker network create my-network

docker run -d --name redis --network backend redis:1.0
docker run -d --name postgresql --network backend -p 5432:5432
postgresql:1.0 docker run -d --name python-app --network backend -p
8080:8080 python-app:1.0 docker run -d --name node --network backend -p
8888:8888 node:1.0
docker run -d --name dotnet--network backend dotnet:1.0
```

## 2. Documentation Docker-Compose

Afin de démarrer le docker-compose, vous devez vous placer à la racine du projet et faire la commande :

```
docker compose up
```

## 3. Documentation docker Swarm

Pour mettre en place notre cluster Docker Swarm, nous sommes partis de notre fichier **docker-compose.yml** existant. L'objectif était de **réadapter les services pour fonctionner en mode Swarm**, tout en conservant les fonctionnalités essentielles telles que la **persistance des données**, la gestion des **réseaux**, et le **load balancing** pour les services exposés.

Nous avons déployés l'ensemble des services sur un seul noeud ici Docker Desktop mais nous avons configuré des réplicas. Ceci permettant dans un cluster multi noeuds, les réplicas se distribueraient sur 1 manager et 2 workers comme demandés

## 1. Adaptations principales

### 1. Alias et réseau overlay

En Swarm, les conteneurs communiquent entre eux via le **nom de service** sur un réseau **overlay**.

Ainsi, dans Node.js ( `server.js` ), la connexion à PostgreSQL se fait via l'alias `db` :

```
const pool = new pg.Pool({
  connectionString: "postgres://postgres:postgres@db/postgres"
});
```

Cela évite d'utiliser des IP fixes et permet aux services de rester accessibles même si les réplicas changent d'hôte.

### 2. Mode deploy et réplicas

Nous avons ajouté le bloc `deploy` pour définir :

- Le nombre de réplicas pour certains services tel que le vote ou result
- La politique de redémarrage en cas d'échec

```
deploy:
  replicas: 2
  restart_policy:
    condition: on-failure
```

### 3. Healthchecks

Nous avons défini des "healthchecks" pour PostgreSQL et Redis afin que les services dépendants ne démarrent qu'une fois ces services prêts.

## 2. Commandes utilisées

- `docker swarm init` : Initialise le Docker Swarm.

- `docker stack deploy -c docker-stack.yml projet-app` : Déploie la stack sur le Swarm.
- `docker stack services projet-app` : Affiche tous les services et leur état.
- `docker service logs projet-app_db` : Permet de vérifier les log de PostgreSQL.
- `docker stack rm projet-app` : Supprime la stack et tous ses services.

### 3. docker-stack.yml

Vous retrouverez ci dessous, le docker-stack.yml utilisé pour le deployment. Il est également disponible dans le projet à la racine.

```
version: "3.9"
services:
# Redis

  redis:

    image: redis:7

    networks:

      - backend

    deploy:

      replicas: 1

      restart_policy:

        condition: on-failure

    healthcheck:

      test: ["CMD", "redis-cli", "ping"]

      interval: 5s

      timeout: 3s

      retries: 5

# Postgres

  db:
```

image: postgres:16

environment:

POSTGRES\_USER: postgres

POSTGRES\_PASSWORD: postgres

POSTGRES\_DB: postgres

volumes:

- pgdata:/var/lib/postgresql/data

networks:

backend:

aliases:

- postgres

deploy:

replicas: 1

placement:

constraints:

- node.role == manager

restart\_policy:

condition: on-failure

healthcheck:

test: ["CMD", "pg\_isready", "-U", "postgres"]

interval: 5s

timeout: 5s

retries: 5

# Python-app

vote:

image: python-app-compose:1.0

networks:

- frontend
- backend

ports:

- target: 8080
- published: 8080
- protocol: tcp
- mode: ingress

environment:

REDIS\_HOST: redis

POSTGRES\_HOST: postgres

deploy:

replicas: 2

restart\_policy:

condition: on-failure

# Node

result:

image: node-compose:1.0

networks:

- frontend
- backend

ports:

- target: 8888

published: 8888

protocol: tcp

mode: ingress

deploy:

replicas: 2

restart\_policy:

condition: on-failure

# Dotnet

worker:

image: worker-compose:1.0

networks:

- backend

environment:

REDIS\_HOST: redis

POSTGRES\_HOST: postgres

deploy:

replicas: 1

restart\_policy:

condition: on-failure

volumes:

pgdata:

networks:

frontend:

driver: overlay

backend:

driver: overlay